

ZJU Summer 2025 Contest 5

Contest Analysis

07.10.2025

F. treasure

Description

有 n 个宝箱和 n 个钥匙, 分布在 x 正半轴上, 一把钥匙只能开一个宝箱, 求从坐标原点出发, 所需要的最小总路程

F. treasure

Solution

首先, 我们将 a, b 排序, 通过交换法可以证明, 若存在一种最优解满足最小总路程, 则可以调整为第 i 个宝箱和第 i 把钥匙配对

那么所有 $a_i \geq b_i$ 的宝箱都可以直接顺手拿 (不用折回去拿), 故可以删掉 (只需要到保证最右边的宝箱和盒子被取到就行).

那么我们可以通过 dp 来求解, 令 f_i 表示拿完了 $[1, i]$ 的宝箱, 目前在 a_i 的最小价值

容易列出状态转移方程式

$$f_i = \min f_j + b_i - a_j + a_i + b_i - 2 \times a_{j+1}$$

可以通过前缀最大值优化, 注意最后一步它不需要折回去, 直接统计就行

D. Explosion

Description

给定一个序列，每次可以选四个数，把中间两个加到旁边的。直到不能操作。问最终序列有几种可能性。

D. Explosion

Solution

考虑对差分序列的影响，则每次操作是合并相邻的两对差分。
不妨假设差分数组中相邻的数之间有一块隔板，则等价于选择隔板 $i-1, i, i+1$ ，并抽掉 $i-1, i+1$ ，问最后剩下可能是哪一个（两个）隔板。
 n 为偶数时答案容易计算，是 $n-3$ 。
 n 为奇数时，枚举两个隔板之间的距离，则只对两端隔板数量的最小值和奇偶性有限制。

A. π -mon Polygon 2

Description

给定一个简单多边形，和大于简单多边形周长的 D ，求随机投掷简单多边形和间距为 D 的直线相交的概率

A. π -mon Polygon 2

Solution

直线和简单多边形相交等价于和其凸包相交，所以先求出简单多边形的凸包。

因为 d 足够大，所以其实我们是要求凸包随机旋转后宽度的期望，可以直接用旋转卡壳计算，较为繁琐。

观察到凸包和直线相交一定有恰好两个交点，只有一个交点的情况出现概率为 0 可以忽略不计，那么我们可以计算每条边和凸包相交的概率之和，除以二就是答案。所以最终答案只和凸包周长有关，假设周长为 C ，那么答案为 $\frac{C}{\pi D}$

C. City

Description

给定若干个数，对每个数求出和他互质的最大的数。

考虑将数组排好序后整体二分。假设当前二分区间为 $[l, r]$ ，当前答案在该区间询问集合为 S ，那么我们需要对 S 的每个数求出是否存在 $(mid, r]$ 中的数与其互质。

由：

$$\sum [\gcd(a, b) = 1] = \sum_{k|a, k|b} \mu(k)$$

我们可以枚举 $(mid, r]$ 中每个数的因子，将贡献塞进桶里，然后再枚举 S 中每个数的质因子，就可以知道有多少个 $(mid, r]$ 中的数与该数互质。
时间复杂度： $O(n \log n 2^\omega)$ 。

P.S. 存在其他各显神通的做法，此处不表。

B. Butterfly

Description

给定 $n \cdot m$ 的方格，黑白染色使得存在至少一个蝴蝶形状，问方案数。

B. Butterfly

Solution

考虑计算不存在蝴蝶形状的方案数。对于每个中间的格子，我们需要钦定一个方向，使得这个方向上只有和他异色的格子。可以发现，对于每一行每一列，对于某一个方向（上下左右），最多只能存在一个这个方向的格子。故最多只有 $2(n-2) + 2(m-2)$ 个格子，即

$2(n-2) + 2(m-2) \geq (n-2)(m-2)$ 时答案才非 0。

不妨令 $n \leq m$ ，则只有 $n \leq 4$ 时才会有充分大的 m ，否则 m 的范围也很小。

B. Butterfly

Solution

考虑直接按行状压 dp，每次逐行转移即可。复杂度根据状压的状态设计不同而不同。 $O(m \cdot 14^n)$ 的复杂度可以通过。

每一列的状态有 9 种本质不同的可能：对于每种颜色，有：不包含这种颜色；包含了，但是之后还可以填；以及之后不允许再填这个颜色。直接枚举每一行转移是 $O(m \cdot 18^n)$ ，可能需要一些优化才能通过。注意到 9 种状态有两种可以规避：两种颜色都不包含，和两种颜色都不允许填。复杂度 $O(m \cdot 14^n)$ 。

E. railway

Description

给定一棵大小为 n 的树的每条边定向，进行 m 次操作，每次反转某条边的方向或查询一条路径上所有点经过有向边能到达的点的并集大小。

E. railway

Solution

任取一个点作为根，记 val_u 为 u 向子树内能到达的点的数量， up_u 表示 u 与其父亲的连边是否向上。

对于一个询问 (u, v) ，记 l 为 $LCA(u, v)$ ， t 为 l 向上能到达的最远点， S 为路径 $u \rightarrow v$ 和 $l \rightarrow t$ 上点的并集，则答案为 $val_t + \sum_{u \in S \ \& \ up_u} val_u$ 。

考虑如何支持修改。假设当前修改的是 u, v 之间的边（设 u 为父亲），记 t 为最远的能到达 u 的祖先，则每次修改会将 t 到 u 的所有点的 val 加（或减） val_v ，即路径修改单点查询。

E. railway

Description

考虑计算答案。再用一个数据结构维护所有 $up = 1$ 的点的 val ，每次修改至多影响两个点 (v 和 t)，计算答案只需路径求和。以上两个部分都可以转化为 dfn 序上的维护。

再考虑如何找到 u 向上能到达的最远点（最远的能到达 u 的祖先同理）。可以在树链剖分时对每个链顶单独维护向下的长度，这样每次修改至多影响一个链顶，查询只需若干次链顶 check 和一次线段树上二分。各个部分都能做到 $\mathcal{O}(n \log n)$ （假设 m, n 同级）。